Project 1: Summarising Medical Text

1. Project Overview

A medical software company would like to introduce a new product for busy radiologists. The company would like to build an app for radiologists to view an accurate text summary of findings from a patient's medical imaging visit. They have a dataset of free-text radiology visit findings and corresponding short summaries. They would like an AI engineer to use the data to build and deploy a prototype text summarization model, taking free-text visit findings and summarizing them into accurate and concise impressions.

Your key tasks are to prepare the data, build a model to summarize radiology report text, evaluate the model performance, deploy it to a live API, and provide appropriate documentation. Your model may be a brand-new model or a fine-tuned adaptation of an existing model. You may use any AI model approach, such as developing a natural language processing (NLP) or deep learning model, or using a large language model (LLM). If you chose to use a pre-trained model such as an LLM, you must fine-tune the model yourself, rather than deploying an unmodified, pre-existing model. Your model must have a minimum ROUGE1 F1 score of 0.3 in order to be useful for the company (note this score is achievable by fitting or fine-tuning a model on a standard consumer PC).

You are expected to use Python for data preparation and model building. For the API deployment, you may use Python or JavaScript. You are not constrained by the contents of the courses, and you may employ any suitable approaches, modelling techniques, packages and libraries to achieve the end result.

2. Minimum skill set

The following skills are required to complete the project:

- Python
    - Data preparation
    - Data cleaning
    - AI modelling techniques
- Python or JavaScript
    - REST APIs

3. Additional resources

Python 3 should be installed from: https://www.python.org/downloads/. A third-party Python IDE such designed for data science as PyCharm Community Edition (free) https://www.jetbrains.com/pycharm/download/other.html can be installed to make package management and debugging easier, but is not required.

If you wish to use JavaScript you may install a third-party IDE designed for web development such as WebStorm (free for non-commercial use) https://www.jetbrains.com/webstorm/, but is not required. You may also optionally use a JavaScript framework such as node.js https://nodejs.org.

You must use an API hosting service to host your live API. For example, free plans are available from https://render.com/ and https://fly.io/, or 6-12 month free trials from Amazon (https://aws.amazon.com/) and Microsoft (https://azure.microsoft.com) – be aware of future billing with free trials (you are not limited to these choices). If using GitHub with render.com for deployment, a large model file may be stored on GitHub with Git Large File Storage.

The data is available from: https://openi.nlm.nih.gov/faq?download=true (select "Where can I get the Chest X-ray images in Open-i?" and then "Reports: Link"). If the data is not available from the website, you may load it from the supplied TAR-GZIP file: "NLMCXR_reports.tgz". The field "FINDINGS" is the free-text findings to summarize. The field "IMPRESSION" is the corresponding summary text. All other fields should be ignored.

The following additional Python packages may be useful, though you are not limited to these:

- pandas (https://pandas.pydata.org/): for data preparation and cleaning
- NumPy (https://numpy.org/): for data preparation and cleaning
- PyTorch (https://pytorch.org/): variety of AI models including LLMs
- TensorFlow (https://www.tensorflow.org/): variety of AI models including LLMs
- Natural Language Toolkit (https://www.nltk.org/): NLP
- Huggingface (https://huggingface.co/docs): complete end-to-end AI model framework
    - Pre-trained models: https://huggingface.co/models
    - Transformers (model definition and fine-tuning framework): https://huggingface.co/docs/transformers/index
    - Tokenizers: https://huggingface.co/docs/tokenizers/index
    - Datasets (data framework): https://huggingface.co/docs/datasets/index
    - Evaluate (model evaluation framework): https://huggingface.co/docs/evaluate/index
- rouge_score (https://pypi.org/project/rouge-score/): ROUGE1 F1 score
- Flask (https://flask.palletsprojects.com/en/stable/): REST API and web apps
- FastAPI (https://fastapi.tiangolo.com/): REST API

4. Outputs

1) LIVE API model deployment

A model on a live, publicly accessible API, which takes a free-text prompt containing radiology report text as an input and outputs a short summary. The model must meet a minimum performance score (see below 'Model Outputs' section).

The model should be deployed into an API, and the live model should correspond to the final version of the model in the supplied source code. The API section of your documentation should correspond to the live API, so that a user can follow along to use your API. The API should contain a single endpoint using the POST method to take the free-text "findings" field in the input request body, and it should return a short free-text summary "impression" field as an output (or an appropriate error message if the "findings" field is not input).

2) Source Code

The source code should be written in Python (you may also use JavaScript for the API piece) and be fully reproducible, e.g. another AI engineer with access to the data could run the code from

end-to-end to produce the deployed model from (1) and the performance results shown in the documentation. It should contain the following sections:

- Data Processing: Process the raw data to create a flat file for modelling.
- Data Cleaning and Feature Creation: If any data cleaning is required (such as missing data), include it here. Create features needed for modelling, such as tokenization.
- Text Summarization Model: Model which takes a free-text radiology findings field and summarizes it into a short "impression". Use best practice for model building. You must include a train-test split of the data. You may include model hyperparameter optimization. You may use any suitable model approach as long as it meets the minimum performance requirements. If you use a pre-existing model such as a LLM, you must fine-tune it using the supplied data, save it locally, and deploy the local version to the API.
- Model Outputs: Make model predictions on the test dataset. Calculate the ROUGE1 F1 score on the test dataset. The model should have a minimum ROUGE1 F1 score of 0.3. Show an example of 3 model predictions, to ensure outputs are fluent and correspond to given text inputs (see "Appendix: Sample Findings for Prediction") for findings to test.
- Model Deployment: A separate file containing the source code used to deploy the final model through a REST API, corresponding to the live API in (1)

3) Documentation

The documentation should contain the following sections (1 pages maximum in total):

- Model summary: High-level one paragraph summary of the modelling method chosen, data preparation and feature creation, model building or fine-tuning approach, hyperparameter optimization (if any), train-test split, and model performance.
- A short paragraph detailing any limitations of your implementation and recommendations for performance improvement. For example, if you fine-tuned a LLM on your local machine, how you reduced the model parameters to allow the model to run, and how you would improve it on a full implementation with multiple GPUs.
- REST API documentation – following the documentation template in the "Appendix: REST API Documentation Template"

5. Project approach

The following approach is suggested to produce the project outputs required from the data, but the final decision is up to you. Start by processing the data, creating a basic working model, and iteratively improving it.

1. Process the raw data into a flat file for modelling
2. Create train/test splits
3. Data cleaning (if required)
4. Feature creation
5. Model building (from scratch, or fine-tuning an existing model)
6. Iteratively improve model performance by re-doing steps (3)-(5) until model performance is sufficient
7. Model performance statistics on test split and sample prediction outputs
8. Deploy model to live REST API

9. Documentation

Appendix: Sample Findings for Prediction

1. The trachea is midline. The cardiomediastinal silhouette is normal. The lungs are clear, without evidence of acute infiltrate or effusion. There is no pneumothorax. The visualized bony structures reveal no acute abnormalities.
2. The lungs are clear. Heart size and mediastinal contours are normal. No osseous abnormalities.
3. AP and lateral views were obtained. Bibasilar atelectasis and small left-sided pleural effusion. Stable cardiomegaly. No pneumothorax. Mild pulmonary vascular congestion.

Appendix: REST API Documentation Template

<1 sentence description of the REST API>

URL: *<url here>*

Method: *<method here>*

Authentication required : *<YES/NO as appropriate>*

Data example

<example API input here>

Success Response

Code : *<success code here>*

Content example

<example success API output here>

Error Response

Condition: *<1 sentence description of condition that triggers the error>*

Code: *<error code here>*

Content:

<failure API output here>